Replication Plan

FDTD: Solving 1+1D Delay PDE in Parallel

OSTI ID: 1475143

Auto-generated Replication Blueprint

April 8, 2026

Contents

Paper Overview

This paper presents a finite-difference time-domain (FDTD) method for solving 1+1D partial differential equations with time delays. The authors develop a parallelization strategy that exploits the causal structure of delay PDEs—the delay term means that future values depend on past values at a fixed lag, enabling a pipeline-parallel approach. The paper demonstrates the method on specific delay PDEs (e.g., delay diffusion equation, delay wave equation), analyzes stability (CFL-like conditions modified for the delay), and benchmarks parallel speedup.

Detailed Workflow

Phase 1: FDTD Scheme Implementation

1.1 Discretize the delay PDE.

- Spatial domain: $x \in [0, L]$ with N_x grid points, spacing $\Delta x = L/N_x$.
- Temporal domain: $t \in [0, T]$ with N_t time steps, spacing Δt.
- Delay: $\tau > 0$ (fixed), requiring storage of solution at times $t - \tau$.

1.2 Implement the FDTD update rule.

- Standard central differences for spatial derivatives.
- Forward Euler or Crank–Nicolson for time stepping.
- Delay term: look up $u(x, t - \tau)$ from stored history buffer.
- Implement boundary conditions (Dirichlet, Neumann, periodic as specified).

1.3 History buffer management.

- Store solution snapshots for $t \in [t_n - \tau, t_n]$; use circular buffer.
- Handle interpolation if $\tau/\Delta t$ is not an integer.

Phase 2: Stability Analysis

2.1 Derive and verify CFL-like stability condition for the delay FDTD scheme.

2.2 Numerically test stability boundaries.

- Run simulations at various $\Delta t/\Delta x$ ratios near the stability boundary.
- Verify that solutions blow up beyond the predicted threshold.

Phase 3: Serial Verification

3.1 Solve test problems with known analytical solutions.

- Delay diffusion equation with manufactured solution.
- Verify convergence order (second-order in space, first/second in time).

3.2 Reproduce solution plots from the paper (space-time contour plots, time traces at fixed x).

Phase 4: Parallelization

4.1 Implement pipeline-parallel FDTD.

- Partition the time domain into blocks of size τ.
- Within each block, the delay term references only the previous block (already computed).
- Use MPI or multiprocessing to assign blocks to different processors.

4.2 Implement spatial-domain decomposition (standard for FDTD) as a comparison.

4.3 Implement hybrid spatial–temporal parallelism if described.

Phase 5: Performance Benchmarking

5.1 Measure wall-clock time vs. number of processors for fixed problem size (strong scaling).

5.2 Measure efficiency as problem size grows with fixed processors (weak scaling).

5.3 Reproduce speedup plots from the paper.

5.4 Compare parallel vs. serial solutions for numerical equivalence (bitwise or within tolerance).

Datasets

All data is synthetically generated by the FDTD solver. No external datasets are needed.

Dataset / Source	Type	Location / Notes
Initial/boundary conditions	Synthetic	Specified analytically in the paper
Manufactured solutions	Analytic	Derived from the PDE for convergence tests
Timing benchmarks	Generated	Collected from parallel runs on the target hardware

Models, Tools, and Software

Programming Languages

- **Python 3.8+** — for prototyping and serial implementation.
- **C/C++ or Fortran** — for performance-critical parallel implementation.

Libraries and Frameworks

Tool / Library	Version	Purpose / URL
NumPy	≥ 1.21	Array operations for serial FDTD
SciPy	≥ 1.7	Sparse matrices, reference ODE solvers for validation

Tool / Library	Version	Purpose / URL
mpi4py	≥ 3.1	MPI bindings for Python parallel implementation `https://mpi4py.readthedocs.io/`
MPI (OpenMPI / MPICH)	system	Message passing for distributed parallelism
Matplotlib	≥ 3.4	Solution visualization, speedup plots
Numba or Cython	latest	Optional JIT compilation for Python FDTD loops
multiprocessing	stdlib	Shared-memory parallelism alternative

Hardware Requirements

- Multi-core workstation or small cluster for parallel benchmarking (4–64 cores).
- RAM: modest ($\sim$4 GB per process) for 1+1D problems.